Knowledge Representation and Reasoning – Mod. 2

Exercise Sheet 3 & 4: RDF & SPARQL – Solutions

Gian Carlo Milanese

27 April/4 May 2023

Exercise 1 RDF Triples

Which of the triples from Fig. 1 are valid? If a triple is not valid, explain why. Validity in this exercise means that the RDF terms (IRIs, literals, blank nodes) are in the correct position. It does not necessarily mean that the triples are good examples of how information should be represented.

	Subject	Predicate	Object	Valid?
1	`<http://example.org/alice>`	`rdf:type`	`foaf:Person`	...
2	`_:alice`	`_:Type`	`_:Person`	...
3	`ex:bob`	`rdf:type`	`foaf:Person`	...
4	`_:Alice`	`rdf:type`	`foaf:Person`	...
5	`"bob"^^xsd:string`	`rdf:type`	`_:Person`	...
6	`_:alice`	`rdf:type`	`"Person"^^xsd:string`	...
7	`_:bob`	`rdf:type`	`_:Person`	...
8	`_:Bob`	`"Type"^^xsd:string`	`foaf:Person`	...

Figure 1: Triples for exercise 1

Solution. 1: valid; 2: not valid (blank node as predicate); 3: valid; 4: valid; 5: not valid (literal as subject); 6: valid; 7: valid; 8: not valid (literal as predicate).

Note that, for example, triple n. 6 is valid (blank nodes can be used in the subject position, and literals in the object position), but it is a very bad example of an RDF triple. It makes no sense to use a blank node to denote Alice (a specific person, so it should be an IRI), and it makes no sense to use a literal to denote the class of all people (should also be an IRI).

The only good triples in this sense are triple n. 1 and triple n. 3.

Exercise 2 RDF Graphs

Write down a Turtle document corresponding to the following descriptions, using explicitly typed literals where relevant.

1. Bob is a student. Bob knows Alice. Bob is 22-years old. Bob is from Milan.
2. Alice is a 23-year old student who likes the band Pink Floyd.

3. Nick Mason is a drummer and a member of the band Pink Floyd. His name is "Nicholas" and he was born on 27 January 1944.
4. The band Pink Floyd has a member whose name is "David Gilmour" and who plays the guitar.
5. Bob knows that Alice likes Pink Floyd.
6. Alice is attending a Polyphia concert on 23 May 2023 in Milan.

You can use prefixes, in particular the ones defined below:

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns##> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema##> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix ex: <http://example.org/> .
## continue here
```

Use an online validator such as the following to ensure that your syntax is valid.

- http://ttl.summerofcode.be/. Turtle validator.
- https://www.easyrdf.org/converter. Here you can convert RDF data between different serializations. An error will be thrown if your syntax is not correct.
- https://www.w3.org/RDF/Validator/. Here you can validate RDF data serialized as RDF/XML (you can use the previous converter to convert it to this format). If you select "Triples and Graph" among the Display Result Options you can also see a graph representation of your RDF triples.

Solution. A possible solution is the following:

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix dbr: <http://dbpedia.org/resource/> .
@prefix dbo: <http://dbpedia.org/ontology/> .
@prefix dbp: <http://dbpedia.org/property/> .
@prefix : <http://example.org/> .

:bob a :Student ;
     foaf:knows :alice ;
     dbo:age "22"^^xsd:integer ;
     dbo:birthPlace "Milano"@it .

:alice dbo:age "23"^^xsd:integer ;
       a :Student ;
       :likes dbr:Pink_Floyd .

dbr:Nick_Mason a :Drummer ;
               foaf:name "Nicholas"^^xsd:string ;
               dbo:birthDate "1944-01-27"^^xsd:date .

dbr:Pink_Floyd dbo:bandMember dbr:Nick_Mason ,
                              [ foaf:name "David Gilmour"^^xsd:string ;
```

```
                                :instrument :guitar ] .

:bob :knows [
  a rdf:Statement ;
  rdf:subject :alice ;
  rdf:predicate :likes ;
  rdf:object dbr:Pink_Floyd
] .

:alice :attend [
  a :Concert ;
  :starring dbr:Polyphia ;
  :date "2023-05-23"^^xsd:date ;
  :location dbr:Milan
] .
```

Exercise 3 Reification Puzzle

Translate the RDF graph represented in Fig. 2 to natural language.

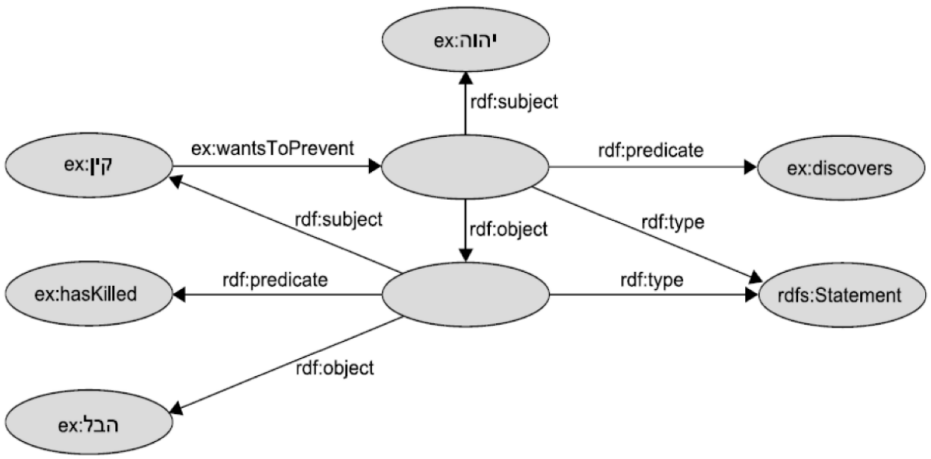

Figure 2: Reification puzzle (source)

Solution. Translation: Cain (קין) wants to prevevent Jehovah (יהוה) from discovering that he (Cain) has killed Abel (הבל).

Exercise 4 SPARQL Basics

Consider the RDF graph of Fig. 3. Give the results tables for the following queries (prefixes are omitted):

1. `select ?x ?y where { ?x :knows ?y . }`

```
@prefix : <http://example.org/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

:bob   :knows :alice .
:carol :knows :alice .

:alice :name "Alice" .
:carol :name "Carol" .

:bob   a :Person .
:carol a :Student .
:david a :MathStudent .

:Student rdfs:subClassOf :Person .
:MathStudent rdfs:subClassOf :Student .

:bob   :occupation :Pianist .
:alice :occupation :Guitarist .

:alice :parentOf :bob .
:bob   :parentOf :carol .
:carol :parentOf :david .

:bob :age 49 .
:alice :age 21 .
:carol :age 33 .
```

Figure 3: Sample RDF Graph for queries

Solution.

x	y
:bob	:alice
:carol	:alice

2. `select ?x where { ?x a :Person . }`

   Solution.

x
:bob

3. `select ?x ?y where { ?x :knows ?y . ?x a :Student .}`

   Solution.

x	y
:carol	:alice

4. `select ?x ?z where { ?x :knows ?y ; a :Student . ?y :name ?z }`

   Solution.

x	z
:carol	"Alice"

5. `select ?x ?y ?z where { ?x :parentOf ?y . OPTIONAL {?y :occupation ?z} }`

Solution.

x	y	z
:alice	:bob	:Pianist
:bob	:carol	
:carol	:david	

6. `select ?x ?y where { ?x rdfs:subClassOf/rdfs:subClassOf ?y }`

Solution.

x	y
:MathStudent	:Person

Exercise 5 SPARQL Playground

Download SPARQL Playground and run it (follow the instructions on the page).

Go to "Data", copy and paste the RDF graph from Fig. 3, click "Reload Turtle Data", then click on "SPARQL Playground". You can now query this RDF graph. Try the following queries:

Solution. Note: prefix definitions will be omitted in the solutions.

- Retrieve all `?x` that know `:alice`.
  Solution.
  ```
  SELECT ?x WHERE { ?x :knows :alice }
  ```
- Retrieve all `?x` and `?y` such that `?x` is the parent of `?y`
  Solution.
  ```
  SELECT ?x ?y WHERE { ?x :parentOf ?y }
  ```
- Retrieve all `?x` and `?y` such that `?x` is the grandparent of `?y`
  Solution.
  ```
  SELECT ?x ?y WHERE { ?x :parentOf/:parentOf ?y }
  ```
- Retrieve all `?x` that know `:alice`, and their name.
  Solution.
  ```
  SELECT ?x ?name WHERE { ?x :knows :alice ; :name ?name }
  ```
- Retrieve all `?x` that know `:alice`, and their name *if available* (hint: use OPTIONAL).
  Solution.
  ```
  SELECT ?x ?name WHERE { ?x :knows :alice .
  optional { ?x :name ?name  } }
  ```
- Retrieve all ancestors (i.e., parents of parents of parents of parents...) of `:david` (hint: use property paths).
  Solution.
  ```
  SELECT ?x WHERE { ?x :parentOf+ :david }
  ```
- Retrieve all `?x` of type `:Person`.
  Solution.
  ```
  SELECT ?x WHERE { ?x a :Person }
  ```
- Retrieve all `?x` of type `:Person`, including subclasses of `:Person`.
  Solution.
  ```
  SELECT ?x WHERE { ?x a/rdfs:subClassOf* :Person }
  ```
- Retrieve all `?x` that are either pianists or guitarists (hint: use UNION).
  Solution.
  ```
  SELECT ?x WHERE {
  { ?x :occupation :Pianist }
  UNION
  ```

```
{ ?x :occupation :Guitarist }
}
```

- Retrieve all `?x` that are older than 30 (hint: use FILTER).
  Solution.

```
SELECT ?x WHERE {
?x :age ?age .
filter (?age > 30 )
}
```

Exercise 6 DBpedia: Modifiers

Go to https://dbpedia.org/sparql, an online SPARQL endpoint where you can query the DBpedia graph.

A few notes:

- The DBpedia SPARQL endpoint uses some default prefixes for IRIs, so you don't need to specify, e.g., `PREFIX dbo: <http://dbpedia.org/ontology/>` in each query. You can see the default prefixes used by this SPARQL endpoint here: https://dbpedia.org/sparql/?help=nsdecl
- If you need to use an IRI and don't know its name, you can try to guess it or Google it (some IRIs will be provided anyway). Note that DBpedia links for resources such as http://dbpedia.org/resource/Nile will redirect you to http://dbpedia.org/page/Nile. You should use `<http://dbpedia.org/resource/Nile>` (abbreviated `dbr:Nile`) in the queries.

Try the following queries:

1. Retrieve 100 triples

   Solution. `select * where { ?s ?p ?o } LIMIT 100`

2. Retrieve 100 distinct triples

   Solution. `select distinct * where { ?s ?p ?o } LIMIT 100`

3. Retrieve 100 distinct predicates.

   Solution. `select distinct ?p where { ?s ?p ?o } LIMIT 100`

4. Retrieve 100 people (i.e., resources of type person)

   Solution. `select distinct ?s where { ?s a dbo:Person } LIMIT 100`

5. Retrieve predicates that can have a resource of type person as subject (some of these predicates are used for later queries)

   Solution. `select distinct ?p where { ?s ?p ?o . ?s a dbo:Person }`

   Or, using `rdfs:domain`: `select distinct ?p where { ?p rdfs:domain dbo:Person }`

6. Retrieve the birth names and years of 100 people

   Solution.

```
select distinct ?n ?y where {
  ?s a dbo:Person ; dbo:birthName ?n ; dbo:birthYear ?y
} order by ?n limit 100
```

7. Retrieve the birth names of 100 people born in 1984, order by name

   Solution.

```
select distinct ?n where {
  ?s a dbo:Person ; dbo:birthName ?n ;
       dbo:birthYear "1984"^^<http://www.w3.org/2001/XMLSchema#gYear>
} order by ?n limit 100
```

8. Retrieve the length of the Nile river.

   Solution. `select ?len where { dbr:Nile dbo:length ?len . }`

9. Retrieve trumpet players that were also bandleaders.

   Solution.

```
select distinct ?uri where {
  ?uri dbo:occupation dbr:Bandleader .
  ?uri dbo:instrument dbr:Trumpet . }
```

10. Retrieve movies that were written, directed and produced by the same person

    Solution.

```
select * where { ?film a dbo:Film ;
                        dbo:director ?dpw ;
                        dbo:producer ?dpw ;
                        dbo:writer ?dpw . }
```

A few useful IRIs:

- dbo:Person
- dbo:length
- dbo:occupation
- dbr:Bandleader
- dbo:instrument
- dbr:Trumpet
- dbo:Film
- dbo:producer
- dbo:writer

Exercise 7 DBpedia: Optional Values

Go to https://dbpedia.org/sparql. Try the following queries:

1. Retrieve names of writers and, if available, their birth year.

   Solution.

```
SELECT ?author_name ?year
WHERE {
  ?author rdf:type dbo:Writer .
  ?author rdfs:label ?author_name .
OPTIONAL { ?author dbo:birthYear ?year } . }
```

2. The same as the previous query but, if available, retrieve their birth place too.

   Solution.

   ```
   SELECT ?author_name ?year ?place
   WHERE {
     ?author rdf:type dbo:Writer .
     ?author rdfs:label ?author_name .
   OPTIONAL { ?author dbo:birthYear ?year } .
   OPTIONAL { ?author dbo:birthPlace ?place } . }
   ```

3. Retrieve movies that were written, directed and produced by the same person. If available, retrieve their box-office gross.

   Solution.

   ```
   SELECT * { ?film a dbo:Film ;
              dbo:director ?dpw ;
              dbo:producer ?dpw ;
              dbo:writer ?dpw .
     optional { ?film dbo:gross ?gross } }
   ```

Useful IRIs:

- `dbo:Writer`
- `rdfs:label` (for names)
- `dbo:birthPlace`
- `dbo:Film`

Exercise 8 DBpedia: UNION

Go to https://dbpedia.org/sparql. Try the following queries:

1. Retrieve movies that were written, directed or produced by Charlie Kaufman

   Solution.

   ```
   select distinct ?x where {
     { ?x dbo:producer dbr:Charlie_Kaufman }
     union
     { ?x dbo:writer dbr:Charlie_Kaufman }
     union
     { ?x dbo:director dbr:Charlie_Kaufman } }
   ```

   or:

   ```
   select distinct * where {
     { ?p dbo:producer dbr:Charlie_Kaufman }
     union
     { ?w dbo:writer dbr:Charlie_Kaufman }
     union
     { ?d dbo:director dbr:Charlie_Kaufman } }
   ```

2. Retrieve people that were born in Milan or in Rome.

   Solution.

```
select distinct * where {
{ ?x dbo:birthPlace dbr:Milan }
union
{ ?y dbo:birthPlace dbr:Rome } }
```

3. Same as the previous query, but also retrieve the occupation for people born in Milan.

   Solution.

   ```
   select distinct * where {
     { ?x dbo:birthPlace dbr:Milan ; dbo:occupation ?z }
     union
     { ?y dbo:birthPlace dbr:Rome } }
   ```

4. Same as the previous query, but retrieve the occupation for people born in Milan only if available.

   Solution.

   ```
   select distinct * where {
     { ?x dbo:birthPlace dbr:Milan optional { ?x dbo:occupation ?z } }
     union
     {?y dbo:birthPlace dbr:Rome} }
   ```

Useful IRIs:

- `dbo:Film`
- `dbr:Milan`
- `dbo:birthPlace`

Exercise 9 DBpedia: Property Paths

Go to https://dbpedia.org/sparql. Try the following queries:

1. Retrieve movies that were written, directed or produced by Charlie Kaufman

   Solution.

   ```
   select distinct ?x where
   { ?x dbo:producer|dbo:writer|dbo:director dbr:Charlie_Kaufman }
   ```

2. Retrieve people that were born or that live in Milan.

   Solution.

   ```
   select distinct * where { ?x dbo:residence|dbo:birthPlace dbr:Milan}
   ```

3. Retrieve people that have written or directed a movie.

   Solution. `select distinct ?w where { ?m dbo:writer|dbo:director ?w }`

4. Retrieve the names of notable works of Elena Ferrante

   Solution.

   ```
   select ?title where {
     dbr:Elena_Ferrante dbo:notableWork/rdfs:label ?title . }
   ```

5. Retrieve the grandparents of Elizabeth II.

   Solution. `select * where { dbr:Elizabeth_II dbo:parent/dbo:parent ?x }`

6. Retrieve all ancestors of Elizabeth II.

   Solution. `select * where { dbr:Elizabeth_II dbo:parent+ ?x }`

7. Retrieve the parents, the grandparents, the great-grandparents, and the great-great-grandparents of Elizabeth II.

   Solution.

```
select * where
{ dbr:Elizabeth_II dbo:parent/dbo:parent?/dbo:parent?/dbo:parent? ?x }
```

A few useful IRIs:

- `dbr:Elena_Ferrante`
- `dbo:notableWork`
- `dbo:residence`
- `dbo:birthPlace`
- `dbr:Elizabeth_II`
- `dbo:parent`

Exercise 10 DBpedia: Ask, Describe and Construct Queries

Go to https://dbpedia.org/sparql. Try the following queries:

1. Was Natalie Portman born in (some city of) the US?

   Solution.

```
ask where {
  dbr:Natalie_Portman dbo:birthPlace ?city .
  ?city dbo:country dbr:United_States .
}
# or
ask where {
  dbr:Natalie_Portman dbo:birthPlace/dbo:country dbr:United_States . }
```

2. Is Rome the capital of Italy?

   Solution. `ask where { dbr:Italy dbo:capital dbr:Rome . }`

3. Describe Rome

   Solution. `describe dbr:Rome`

4. Describe 20 people born in Italy

   Solution. `describe ?x where { ?x dbo:birthPlace dbr:Italy } limit 20`
   or
   `describe ?x where { ?x dbo:birthPlace/dbo:country dbr:Italy } limit 20`
   or
   `describe ?x where { ?x dbo:birthPlace/dbo:country? dbr:Italy } limit 20`

5. Construct the graph of the ancestors of `dbo:Elizabeth_II`

   Solution.

```
construct { ?x dbo:parent ?y }
where {  ?x dbo:parent ?y . dbr:Elizabeth_II  dbo:parent* ?x }
```

6. Construct an RDF graph about characters from "A Song of Ice and Fire" (or some other series you like) and their relatives (and/or some other properties).

   Solution.

   ```
   construct { ?x dbo:relative ?y } where {
     ?x dbo:relative ?y ; dbo:series dbr:A_Song_of_Ice_and_Fire }
   ```

   or

   Solution.

   ```
   construct { ?y ?p ?x } where
     { values ?p {dbo:relative dbp:children dbp:origin dbo:spouse dbp:alias}
      ?y ?p ?x .
     { ?y dbo:series dbr:A_Song_of_Ice_and_Fire }
     union
     { ?y dbo:series dbr:Game_of_Thrones } }
   ```

   Read here about the `values` keyword.

A few useful IRIs:

- `dbr:Natalie_Portman`
- `dbo:birthPlace`
- `dbo:country`
- `dbr:United_States`
- `dbo:capital`
- `dbo:parent`
- `dbo:relative`
- `dbo:series`
- `dbr:A_Song_of_Ice_and_Fire`

Exercise 11 DBpedia: Filters

Lists of functions for reference:

- On RDF terms: https://www.w3.org/TR/sparql11-query/#func-rdfTerms
- On strings: https://www.w3.org/TR/sparql11-query/#func-strings
- On numerics https://www.w3.org/TR/sparql11-query/#func-numerics
- On dates ad times https://www.w3.org/TR/sparql11-query/#func-date-time

Go to https://dbpedia.org/sparql. Try the following queries:

1. Retrieve the names of 100 people born after 1984

   Solution.

   ```
   select distinct ?n ?b where {
     ?s a dbo:Person ; dbo:birthName ?n ; dbo:birthYear ?b .
     filter (?b > "1984"^^<http://www.w3.org/2001/XMLSchema#gYear>)
   } order by ?n limit 100
   ```

2. Retrieve writer names (in English), the title of their works (in Italian) and their page number, but only if the page number is either less than 100 or over 9000.

   Solution.

```
SELECT ?author_name ?title ?pages
WHERE {
  ?author rdf:type dbo:Writer .
  ?author rdfs:label ?author_name .
  ?author dbo:notableWork ?work .
  ?work rdfs:label ?title .
  ?work dbo:numberOfPages ?pages
  FILTER ((?pages < 100 || ?pages > 9000)
           && lang(?title)="it"
           && lang(?author_name) = "en") .
  ?work rdfs:label ?title .
}
```

3. Retrieve movies directed by Alfred Hitchcock that do not star James Stewart.

   Solution.
   ```
   select * where {
     ?m a dbo:Film ; dbo:director dbr:Alfred_Hitchcock .
     filter not exists { ?m dbo:starring dbr:James_Stewart } }
   ```

   or

   ```
   select * where {
     ?m a dbo:Film ; dbo:director dbr:Alfred_Hitchcock .
     minus { ?m dbo:starring dbr:James_Stewart } }
   ```

4. Retrieve movies directed by Alfred Hitchcock that do not star James Stewart and Grace Kelly.

   Solution.
   ```
   select * where {
     ?m a dbo:Film ; dbo:director dbr:Alfred_Hitchcock .
     filter not exists
       { ?m dbo:starring dbr:James_Stewart, dbr:Grace_Kelly } }
   ```

   or

   ```
   select * where {
     ?m a dbo:Film ; dbo:director dbr:Alfred_Hitchcock .
     minus { ?m dbo:starring dbr:James_Stewart, dbr:Grace_Kelly } }
   ```

5. Retrieve movies directed by Alfred Hitchcock that do not star James Stewart or Grace Kelly.

   Solution.
   ```
   select * where {
     ?m a dbo:Film ; dbo:director dbr:Alfred_Hitchcock .
     filter not exists { ?m dbo:starring dbr:James_Stewart }
     filter not exists { ?m dbo:starring dbr:Grace_Kelly }}
   ```

   or

   ```
   select * where {
     ?m a dbo:Film ; dbo:director dbr:Alfred_Hitchcock .
     minus { ?m dbo:starring dbr:James_Stewart }
     minus { ?m dbo:starring dbr:Grace_Kelly } }
   ```

or

```
select * where {
  ?m a dbo:Film ; dbo:director dbr:Alfred_Hitchcock .
  filter not exists { { ?m dbo:starring dbr:James_Stewart }
                      union
                      { ?m dbo:starring dbr:Grace_Kelly } } }
```

or

```
select * where {
  ?m a dbo:Film ; dbo:director dbr:Alfred_Hitchcock .
  minus { { ?m dbo:starring dbr:James_Stewart }
          union
          { ?m dbo:starring dbr:Grace_Kelly }} }
```

A few useful IRIs:

- dbo:starring
- dbr:James_Stewart

Exercise 12 Assignment and Aggregate Queries

1. Find out the number of books written by each writer (hint: group by writer, and use the aggregate function COUNT). Sort the results by number of books written .

   Solution.

   ```
   SELECT ?author (COUNT(?work) as ?bookcount)
   WHERE {
     ?author rdf:type dbo:Writer .
     ?work dbp:author ?author .
   } GROUP BY ?author order by desc(?bookcount)
   ```

2. Find out the average movie runtime by genre. Sort the results by runtime.

   Solution.

   ```
   select ?g avg(?r) as ?avgruntime where
   { ?m a dbo:Film ; dbo:runtime ?r ; dbp:genre ?g }
   group by ?g order by desc(?avgruntime)
   ```

3. Find out how many movies are there (in DBpedia) by genre. Sort the results by number of movies.

   Solution.

   ```
   SELECT ?g count(?movie) as ?numm
   WHERE {
     ?movie  a dbo:Film ; dbp:genre ?g
   } group by ?g order by desc(?numm)
   ```

4. Find out how many movies are there (in DBpedia) by country. Sort the results by number of movies

Solution.

```
    SELECT ?c count(?movie) as ?numm
WHERE {
  ?movie  a dbo:Film ; dbp:country ?c
} group by ?c order by desc(?numm)
```

5. Find out the countries where movies directed by Alfred Hitchcock were made, and how many movies per country.

   Solution.

```
    SELECT ?c count(?movie) as ?numm
WHERE {
  ?movie  dbo:director dbr:Alfred_Hitchcock ; dbo:country ?c
} group by ?c
```

A few useful IRIs:

- `dbp:author`
- `dbo:runtime`
- `dbp:genre`
- `dbp:country`

Answers to a few questions from last year

Are the double quotes part of a literal? The double quotes `""` are not part of a literal. E.g., the literal `"Bob"^^xsd:string` has literal form `Bob` (a string of three characters) and datatype IRI `xsd:string`. This is analogous to e.g. Python where the double quotes are not part of a string like `"abcde"`.

Why query (3) of exercise 6 does not (always) return 100 predicates? The reason why the DBpedia SPARQL endpoint might not return 100 predicates is that the query execution might time out before finding all of them, and then return just a subset. Adjusting the execution timeout parameter might help (although the query execution time might be slower).

Thanks Renzo Alva Principe for providing this solution.

UNION queries with different variables and results table. An example with some simple data should clarify.

Data:

```
@prefix ex: <http://example.org/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

ex:bob   ex:birthPlace ex:Milan ; a ex:Person .
ex:alice ex:birthPlace ex:Rome  ; a ex:Person .
```

With the query

```
prefix ex: <http://example.org/>
select distinct * where {
  { ?x ex:birthPlace ex:Milan }
  union
```

```
  { ?x ex:birthPlace ex:Rome }
}
```

we obtain:

x
ex:bob
ex:alice

With the query

```
prefix ex: <http://example.org/>
select distinct * where {
  { ?x ex:birthPlace ex:Milan }
  union
  { ?y ex:birthPlace ex:Rome }
}
```

we obtain:

x	y
ex:bob	
	ex:alice

With the query

```
prefix ex: <http://example.org/>
select distinct * where {
  ?x a ex:Person .
  { ?x ex:birthPlace ex:Milan }
  union
  { ?y ex:birthPlace ex:Rome }
}
```

we obtain:

x	y
ex:bob	
ex:alice	ex:alice
ex:bob	ex:alice

since

- For the first solution, `ex:bob` satisfies both `?x a ex:Person` and the first disjunct of the union
- In the second solution, `?x a ex:Person` is satisfied by binding `?x` to `ex:alice`, and the second disjunct of the union is satisfied by binding `?y` to `ex:alice`
- In the third solution, `?x a ex:Person` is satisfied by binding `?x` to `ex:bob`, and the second disjunct of the union is satisfied by binding `?y` to `ex:alice`

With the query

```
prefix ex: <http://example.org/>
select distinct * where {
  ?x a ex:Person . ?y a ex:Person .
  { ?x ex:birthPlace ex:Milan }
  union
  { ?y ex:birthPlace ex:Rome }
}
```

we obtain:

x	y
ex:bob	ex:alice
ex:bob	ex:bob
ex:alice	ex:alice

since

- In the first solution, `ex:bob` and `ex:alice` satisfy `?x a ex:Person` and `?y a ex:Person`, respectively, and the union can be satisfied by either the first or second disjunct (without the `DISTINCT` modifier, we would get this solution twice).
- In the second solution, binding both `?x` and `?y` to `ex:bob` satisfies `?x a ex:Person`, `?y a ex:Person` and the first disjunct of the union.
- In the third solution, binding both `?x` and `?y` to `ex:alice` satisfies `?x a ex:Person`, `?y a ex:Person` and the second disjunct of the union.

In order to get people born in Milan and Rome in two different columns, while still explicitly specifying that they must be `ex:Persons`, we can use the query

```
prefix ex: <http://example.org/>
select distinct * where {
  { ?x a ex:Person ; ex:birthPlace ex:Milan }
  union
  { ?y a ex:Person ; ex:birthPlace ex:Rome } }
```

which returns

x	y
ex:bob	
	ex:alice

You can try out these queries with SPARQL Playground.

Acknowledgements

- The reification puzzle of Fig. 2 was taken from this set of slides by Werner Nutt.
- Some of the queries were taken or adapted from Blerina Spahiu's Exercises in RDF & SPARQL for the Data Semantics course of the University of Milano-Bicocca.